ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/268221038

Stock Trading With Random Forests, Trend Detection Tests and Force Index Volume Indicators

Conference Paper · June 2013

DOI: 10.13140/2.1.2756.8324

CITATIONS
10

READS
6,584

3 authors:

Piotr Ładyżyński
Warsaw University of Technology
5 PUBLICATIONS **63** CITATIONS

SEE PROFILE

Kamil Żbikowski
Warsaw University of Technology
11 PUBLICATIONS **180** CITATIONS

SEE PROFILE

Przemyslaw Grzegorzewski
Systems Research Institute of the Polish Academy of Sciences and Faculty of Math…
162 PUBLICATIONS **3,202** CITATIONS

SEE PROFILE

Some of the authors of this publication are also working on these related projects:

Project Fuzzy semi-numbers View project

All content following this page was uploaded by Piotr Ładyżyński on 14 November 2014.

The user has requested enhancement of the downloaded file.

Stock Trading With Random Forests, Trend Detection Tests and Force Index Volume Indicators

Piotr Ładyżyński[(1)], Kamil Żbikowski[(2)], Przemysław Grzegorzewski[(1,3)]

[(1)]Faculty of Mathematics and Computer Science
Warsaw University of Technology
Koszykowa 75, 00-662 Warsaw, Poland
[(2)]Faculty of Electronics and Information Technology
Warsaw University of Technology
Plac Politechniki 1, 00-661 Warsaw, Poland
[(3)]Systems Research Institute, Polish Academy of Sciences
Newelska 6, 01-447 Warsaw, Poland
{p.ladyzynski,pgrzeg}@mini.pw.edu.pl,kamil.zbikowski@ii.pw.edu.pl

Abstract. The goal of this paper is to investigate if the strong machine learning technique is able to retrieve information from past prices and predict price movements and future trends. The architecture of the system with the on-line adaptation ability to non-stationary two dimensional mixed Black-Scholes Markov time series model is presented. The methodology of investment strategies performance verification is also proposed.

Keywords: Stock trading, random forest, trend detection test, financial time serie, quant fund, investment strategies backtesting.

1 Introduction

Since the early beginning of financial markets investors, speculates and gamblers try to predict stocks price movements and dream on the universal trading strategy generating outstanding profits. Many research have been done in the field and the annual sales of trading courses and tutorials are estimated in billion of dollars. However, a little has been done in the area of scientific research related to that field and the most of the described strategies are not verified at real financial markets.

During last years we have conducted some research of algorithmic trading techniques and investigated a lot of strategies proposed in the literature, tutorials for broker clients and trading courses. To our surprise none of these strategies was able to generate profitable signals while tested by the walk forward method, described in this paper.

A lot of publications reveal serious lacks in verification methodology and some of them seem to be only marketing tricks. There are some papers investigating usability of the machine learning techniques for automated stock trading,

but none of them incorporates the walk forward testing on large amount of observations (see [3], [5], [8], [9]). It is obvious that even if a system is tested on out-of sample data (but without walk forward testing), it is easy to choose such a period that the system gives outstanding results. Therefore, although the papers mentioned above consider interesting data mining systems, it is highly probable that walk forward testing would reveal their weakness in long term trading at the real market.

Many research show that time series created by prices change their structure in time and reveal strong non-stationary behavior. Therefore, many classical time series models (like ARCH, AR etc.) are useless for the analysis of real price movements (of course, it does not contradict that some time series model are brilliant for other features of price trajectory estimation, like GARCH model for market volatility). The major purpose of this paper is to make a step towards to answer the question if one of the strongest classifiers, known in the literature, is able to generate profitable trading signals at real market.

In this paper we propose an architecture of the system which monitors the behavior of the upcoming data and prove its ability of on-line adaptation to non-stationary time series. The performance of the system is examined using large scale historical data from S&P 500 index by the moving window walk forward methodology, which was never done in the past. To be more precise, in our approach the trading system is examined by the walk forward testing on 50 randomly chosen shares from S& P 500 index during the 7 years period (2004-2011). We use one hour aggregates (all previous authors used one day aggregates) so the systems is verified on $50 \times 15000 = 750000$ stock quotations. The authors of the most trading publication verify their systems on one to five years period using day aggregation, so their data range is about $5 \times 250 = 1250$. Unfortunately, a verification of the system on such short period is not reliable. Moreover, some authors pre-tune their systems to the test data set (see, e.g. [8]). The drawbacks of such verification are obvious and described in section 3.

Hence the paper deals with extremely interesting and extensively examined hypothesis whether the machine learning techniques is able to capture the market structure for constructing a profitable trading strategy. As it is known from the history of finance that famous traders become famous due to trading books they have written after loosing large amount of money. One may ask if any human being is able to predict permanently price movements or maybe market - by the definition - is a chaotic system only with the poor property of hetreoskedasticity.

2 System architecture

Many investors believe that certain combination of technical analysis indicators contain an important clue for future price movements prediction. Such indicators are designed to aggregate some special features of financial time series. We would try to check if the strong machine learning technique is able to retrieve trading patterns from preprocessed by indicators past prices and volume.

2.1 Trend detection test

In the most trading systems early trend detection is crucial. Let $(X_i)_{i=1}^n$ be a sequence of prices and let

$$m_i = \frac{X_i - X_{i-1}}{X_{i-1}} \tag{1}$$

denote the rate of returns ($i = 1, \ldots, n$). We try to detect trend applying the exponential averaging to the time series of rate of returns, i.e.

$$m_{i+1}^{(exp)} = \lambda m_{i+1} - (1-\lambda) m_i^{(exp)}, \tag{2}$$

where $\lambda = \frac{2}{n+1}$ is a smoothing parameter. The greater values of λ the lesser weights of the observations from the past and greater weights of the recent observations.

The test statistic for trend detection is than defined as follows (see [7])

$$T^{(exp)} = m_n^{(exp)}. \tag{3}$$

The null hypothesis distribution of (3) is calculated numerically using the Monte Carlo method.

2.2 Indicators

The aggregation operators used in technical analysis are called *indicators*. We may distinguish three main groups of indicators: trend following indicators, oscillators and volume indicators.

Trend following indicators Trend following indicators capture trend after it occurs on the market. Investors believe that the current trend continuation is more probable than the reversal trend and make orders according to the direction of the trend. The most important trend following indicators are:

- $p_{k,n}^{val}$, i.e. p-value of trend detection test statistics (3) calculated for day k and rate of returns window of length $n+1$;

- the simple *moving average* of rate of returns in window of length d calculated for day k, i.e.

$$MA_k^d = \frac{1}{d} \sum_{i=0}^{d} m_{k-i}. \tag{4}$$

Oscillators Oscillators try to guess the future trend direction before it occurs on the market. One of the most popular and useful oscillator is the *relative strength index* defined by

$$RSI_k^d = 100 - \frac{100}{1 + RS_k^d}, \tag{5}$$

where

$$RS_k^d = \frac{\sum_{i=0}^{d-1} (X_{k-i} - X_{k-i-1})\mathbb{I}_{(x\geq 0)}(X_{k-i} - X_{k-i-1})}{-\sum_{i=1}^{d-1} (X_{k-i} - X_{k-i-1})\mathbb{I}_{(x<0)}(X_{k-i} - X_{k-i-1})}. \tag{6}$$

By dividing positive and negative price differences the relative strength index tries to capture a market strength.

Another interesting operator is the *Williams oscillator*, which tries to capture the range of the past prices in comparison with the current one, i.e.

$$WL_k^d = 100\frac{\max\{X_k, X_{k-1}, \ldots, X_{k-d-1}\} - X_k}{\max\{X_k, X_{k-1}, \ldots, X_{k-d-1}\} - \min\{X_k, X_{k-1}, \ldots, X_{k-d-1}\}}. \tag{7}$$

Volume indicators Volume indicators analyze additional information about the market condition – a transaction volume. By combining price differences with volume levels we are able to construct operators indicating current demand and supply on the market. Let define

The *force index*, introduced by Alexander Elder (see [1]), is given by

$$FI_k = \lambda FI_k^1 + (1-\lambda)FI_{k-1} \tag{8}$$

where

$$FI_k^1 = vol_k(X_k - X_{k-1}) \tag{9}$$

and where traditionally $\lambda = \frac{2}{14}$. Thus, (8) operator indicates market's overbought and oversold moment.

2.3 Forest learning

The essence of the proposed system is the application of random forests for constructing regression function predicting maximum and minimum prices in the next 28 hours (i.e. 4 trading days).

Let us define the *maximum relative rate of return* H_{ri}^n and the *minimum relative rate of return* L_{ri}^n in next n periods, respectively,

$$H_{ri}^n = \frac{X_i - H_i^n}{X_i} \tag{10}$$

$$L_{ri}^n = \frac{X_i - L_i^n}{X_i}, \tag{11}$$

where $H_i^n = \max\{X_{i+1}, X_{i+2}, \ldots, X_{i+n}\}$ and $L_i^n = \min\{X_{i+1}, X_{i+2}, \ldots, X_{i+n}\}$.

Our goal is to find functions f_l, f_h predicting future price levels, i.e.

$$\begin{aligned}\hat{H}_{ri}^n = f_h(\ & x_i^{(1)}, x_{i-1}^{(1)}, \ldots, x_{i-k}^{(1)}, \\ & x_i^{(2)}, x_{i-1}^{(2)}, \ldots, x_{i-k}^{(2)}, \\ & \ldots \\ & x_i^{(n)}, x_{i-1}^{(n)}, \ldots, x_{i-k}^{(n)})\end{aligned} \tag{12}$$

$$\begin{aligned}\hat{L}^n_{ri} = f_l(\ & x^{(1)}_i, x^{(1)}_{i-1}, \dots, x^{(1)}_{i-k}, \\ & x^{(2)}_i, x^{(2)}_{i-1}, \dots, x^{(2)}_{i-k}, \\ & \dots \\ & x^{(n)}_i, x^{(n)}_{i-1}, \dots, x^{(n)}_{i-k}),\end{aligned} \tag{13}$$

where i stands for the current hour and $x^{(s)}_l$ is a value of indicator s at hour l $(l < i)$.

The structure presented above provides the classifier with the market snapshots obtained for time moments $t = i, i-1, \dots, i-k$.

To solve a regression problem we use the following random forest algorithm (see [4]):

1. From $b = 1$ to B:
   (a) Draw a bootstrap sample $\mathbb{Z}^*$ of size N from the training data.

   (b) Grow a random-forest tree T_b on bootstrapped sample by recursively repeating the following steps for each node of the tree, until minimum size of the node size n_{min} is reached:
      i. Select m variables at random from p variables from feature sample.

      ii. Pick the best variable/split-point among m.

      iii. Split the node into two child nodes.
2. Output the forest: $\{T_b\}^B_{b=1}$

To predict a new point x we get

$$\hat{f}^B_{rf}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x) \tag{14}$$

Now, using the p-value of the trend detection test for three different time horizons, and technical analysis indicators, we construct feature matrices for high and low prices regression models:

$$\begin{array}{cccccc} y & \bar{x} & & & \\ H^n_{rk} & r_k & r_{k-1} & \dots & r_{k-n} \\ & p^{val}_{k,n_1} & p^{val}_{k-1,n_1} & \dots & p^{val}_{k-n,n_1} \\ & p^{val}_{k,n_2} & p^{val}_{k-1,n_2} & \dots & p^{val}_{k-n,n_2} \\ & p^{val}_{k,n_3} & p^{val}_{k-1,n_3} & \dots & p^{val}_{k-n,n_3} \\ & MA^{n_4}_k & MA^{n_4}_{k-1} & \dots & MA^{n_4}_{k-n} \\ & RSI^{n_5}_k & RSI^{n_5}_{k-1} & \dots & RSI^{n_5}_{k-n} \\ & WL^{n_6}_k & WL^{n_6}_{k-1} & \dots & WL^{n_6}_{k-n} \\ & FI_k & FI_{k-1} & \dots & FI_{k-n} \\ & vol_k & vol_{k-1} & \dots & vol_{k-n} \\ & H^n_{r,k-n} & H^n_{r,k-n-1} & \dots & H^n_{r,k-2n} \end{array} \tag{15}$$

$$
\begin{array}{cccccc}
y & \bar{x} & & & \\
L^n_{rk} & r_k & r_{k-1} & \dots & r_{k-n} \\
 & p^{val}_{k,n_1} & p^{val}_{k-1,n_1} & \dots & p^{val}_{k-n,n_1} \\
 & p^{val}_{k,n_2} & p^{val}_{k-1,n_2} & \dots & p^{val}_{k-n,n_2} \\
 & p^{val}_{k,n_3} & p^{val}_{k-1,n_3} & \dots & p^{val}_{k-n,n_3} \\
 & MA^{n_4}_k & MA^{n_4}_{k-1} & \dots & MA^{n_4}_{k-n} \\
 & RSI^{n_5}_k & RSI^{n_5}_{k-1} & \dots & RSI^{n_5}_{k-n} \\
 & WL^{n_6}_k & WL^{n_6}_{k-1} & \dots & WL^{n_6}_{k-n} \\
 & FI_k & FI_{k-1} & \dots & FI_{k-n} \\
 & vol_k & vol_{k-1} & \dots & vol_{k-n} \\
 & L^n_{r,k-n} & L^n_{r,k-n-1} & \dots & L^n_{r,k-2n}
\end{array}
\tag{16}
$$

where r stands for the rate of return, p^{val} denotes p-value, MA is a moving average of the rate of return, RSI is the relative strength index, WL denotes the Williams oscillator, FI is the force index, and vol stands for the volume.

We assume the following values of the parameters representing various time horizons: $n_1 = 7$, $n_2 = 14$, $n_3 = 35$, which correspond to one day, 2 days and 5 days (i.e. a working week), respectively. On the other hand $n_4 = 21, n_5 = n_6 = 7$ are parameters often used for oscillators.

As it is a single snapshot window is described by 10 parameters. Since we take snapshots for $n = 28$ hours (i.e. 4 working days), the single input for the classifier is 280 dimensional.

2.4 Transactional rule

Suppose that $\hat{L}^r_k$, $\hat{H}^r_k$ are the predicted low and high relative rates of return obtained from our model. The main concept of trading is to buy when prices are close to the low boundary and sell when the current price is close to the predicted maximum price value in next 28 hours. This idea might be described by the following algorithm:

if position==NONE AND $\hat{L}^r_k$ is in uptrend **then**
  if $\frac{|C_i - C_i \hat{L}^r_k|}{C_i} \leq \sigma$ **then**
    BUY
    buyPrice=Ci
    position=LONG
  end if
else
  if position==LONG AND $\hat{H}^r_k$ is in downtrend **then**
    if $\frac{|C_i - C_i \hat{H}^r_k|}{C_i} \leq \sigma$ **then**
      SELL
      sellPrice=Ci
      position=NONE
    end if

```
    end if
  end if
  if position==LONG AND (buyPrice−Ci)/buyPrice >= θ then
    SELL
    sellPrice=Ci
    position=NONE
  end if
```

Here, constant C_i-denotes the close price. Parameters of the transactional rule are set to $\sigma = 0.05$ and $\theta = 0.07$. The last IF clause realizes the stop loss condition. A trend existence in predicted values $\hat{L}_k^r$ and $\hat{H}_k^r$ is tested by with test (3).

3 Walk forward testing

It is crucial for trading algorithms to backtest them in a proper way. In particular, it means that data used for training and testing should be separated. Testing on the sample which is used also for training may easily lead to the overfitting. It is very important to validate a model appropriately on the out-of-sample data.

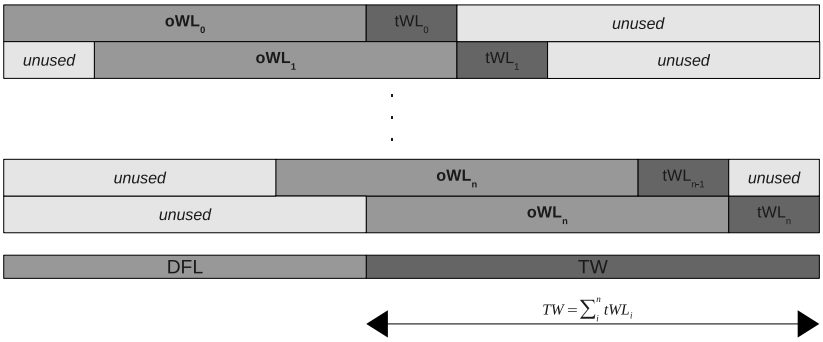

Fig. 1: Walk forward analysis

A classical approach for the model validation is to divide data into three parts used for training, testing and validation. However, this schema is not valid for the financial data where face non-stationary time series. For such class of problems the so-called walk forward analysis can be applied. Its goal is to minimize the curve fitting on the out-of-sample data by shifting the moving window. The window is divided into two subsets. First oWL examples are used for estimating the model. Next, the model is tested on the data consisted of tWL samples. After that, the moving window is shifted by the tWL samples and the next iteration begins. The whole process is illustrated in Figure 1, where DFL denotes the data feed length while TW means the testing window.

Our trading algorithm in it's base version did not manage to beat the market, although it adapted to the data generated from proposed mathematical model. Therefore we decided to proceed additional extensive test. Grid search optimization of free parameters σ, ρ, ϕ and θ were conducted to achieve better performance.

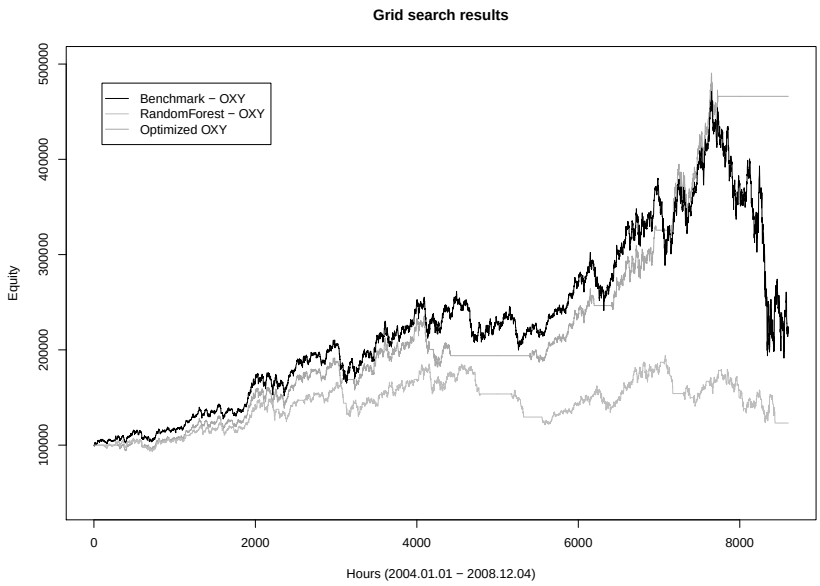

Fig. 2: Results of extensively optimized trading model

Results are shown in Figure 2. Algorithm earned 466119$ while benchmark ended up with nearly half of it. By applying the grid search model fitted the data and successfully followed the benchmark during bull market time. Moreover, it preserved earned money during consolidation phase and bear market times. For us the only conclusion that can be drawn from this experiment is that when someone search for too long, he will do data-snooping, like, on purpose, we did hear. With enough number of free parameters we can fit any data, especially using robust techniques, like one presented in this paper.

4 System backtesting results

The proposed system was verified by performing simulated trading of fifty randomly chosen shares from S&P500 index within dates 1'th of January 2004 to 8'th of August 2011. Average amount aggregates in single trajectory was equal to 150000. System was tested with walk forward methodology with $oWL = 1000$

and $tWL = 30$. Performing backtesting took 35 days of computations using 3-servers powerful cluster (see Fig. 3).

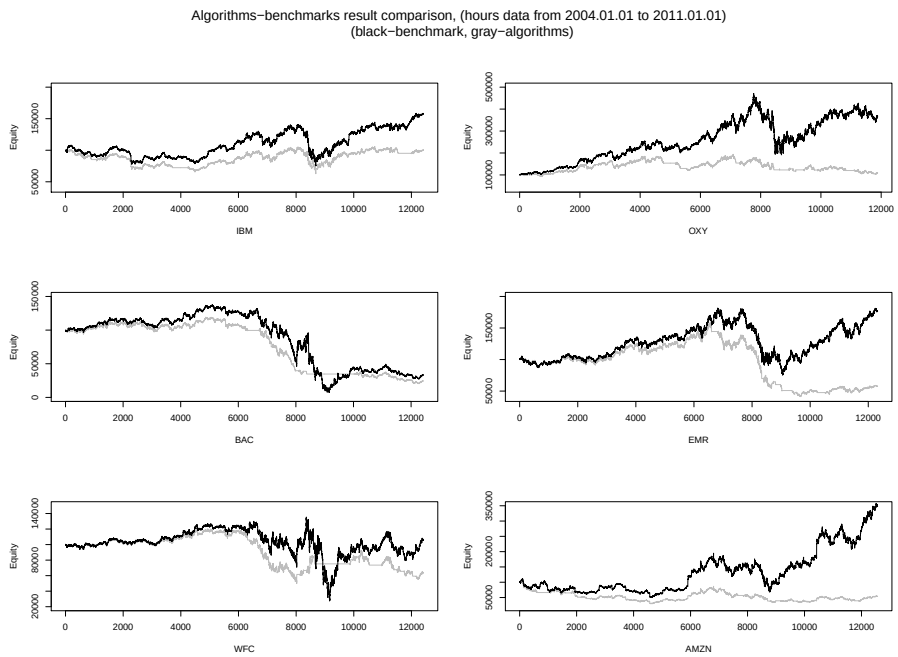

Fig. 3: Model performance trading sample shares from SP500 index

Fig. 4 shows accumulated from all trajectories weekly rate of returns histograms from benchmark and model. Expected value of week return from random share was equal to 0.002766 with corresponding weekly rate of return from model equal to -0.000597. As we can see from Fig. 3 and Fig. 4 buy and hold strategy turned out to be more profitable.

The total MSE of H^n_{ri} and L^n_{ri} prediction was equal to 0.019 0.021, respectively.

To check if the model have any predicting abilities we have constructed the following measures representing the average distance between the close price C_i at hour i and the maximum price in next 28 days, expressed in terms of the rate of return:

$$MSE_{(H^n_{ri},C)} = \sqrt{\sum_{i=28}^{N} \frac{1}{N-28} \left(\frac{max(C_{i-27}, C_{i-23}, \ldots, C_i) - C_{i-28}}{C_{i-28}} \right)^2} \quad (17)$$

$$MSE_{(L^n_{ri},C)} = \sqrt{\sum_{i=28}^{N} \frac{1}{N-28} \left(\frac{min(C_{i-27}, C_{i-23}, \ldots, C_i) - C_{i-28}}{C_{i-28}} \right)^2}. \quad (18)$$

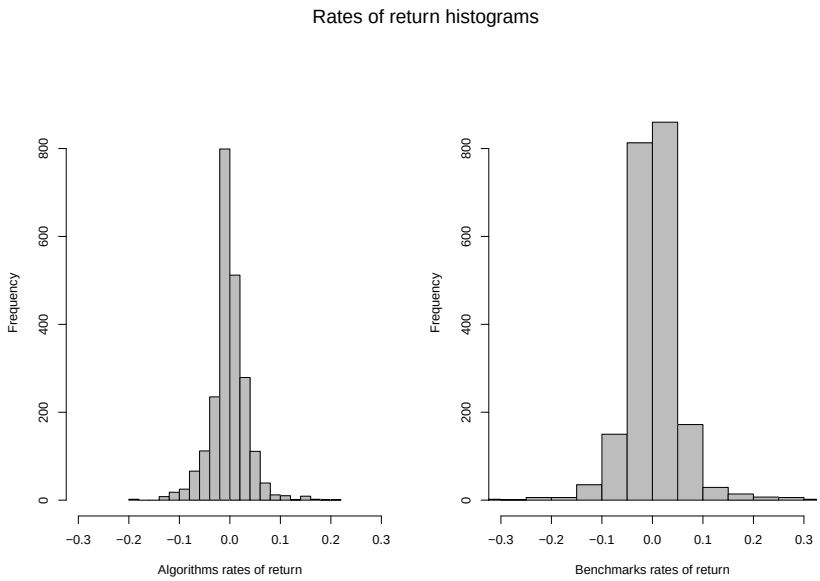

Fig. 4: Weekly rate of returns histograms from benchmark and model (50 randomly chosen shares from SP500)

If the model has no predicting power then these measures should have the same or greater rank than prediction MSE. We computed that $MSE_{(H^n_{ri},C)} = 0.024$, $MSE_{(L^n_{ri},C)} = 0.036$ what is slightly less than model MSE. It would be an interesting issue to formulate and verify precise statistical hypothesis if the model applied for price time series have any predicting power.

5 On-line adaptation to non-stationary two-dimensional mixed Black-Scholes Markov time series model

Although the performance of the system on the real market is not satisfactory it is worth investigating its ability to adapt to non-stationary time series generated from a given model. Let $(S_i, Y_i)_{i=0}^n$ be a two dimensional time series. Let $(Y_i)_{i=0}^n$ be a discrete Markov process with the transition matrix

$$p(k,l) = P(Y_i = k | Y_{i-1} = l) = \begin{bmatrix} \frac{149}{150} & \frac{1}{150} \\ \frac{1}{70} & \frac{69}{70} \end{bmatrix}, \tag{19}$$

where $k, l = 0, 1$. Process Y has only two states and it simulates an important market indicator. Y proceeds serious price movements of the Black-Scholes trajectory S_i. $Y = 0$ - indicates an uptrend. If Y switches to 1, five hours later we have a 25% price downfall and uptrend turns into downtrend. On the other hand if Y switches from 1 to 0, then five hours later a downtrend turns into uptrend.

Let $S_0 = 100$ denote the initial price, $Y_0 = 0, a_0 = 1$ - initial trend (uptrend), $r = 0.005$, $\sigma = 0.002$. The following equations define dependent on Y_i price process S_i:

$$S_i = \begin{cases} S_{i-1} exp((a_i r - \frac{\sigma^2}{2}) + \sigma \mathbb{Z}_i), & \text{if } \mathbb{Y}_{i-5} = 0 \\ 0.75 S_{i-1}, & \text{if } \mathbb{Y}_{i-5} = 1 \end{cases} \tag{20}$$

$$a_i = \begin{cases} a_{i-1}, & \text{if } \mathbb{Y}_{i-5} = 0 \\ -a_{i-1}, & \text{if } \mathbb{Y}_{i-5} = 1. \end{cases} \tag{21}$$

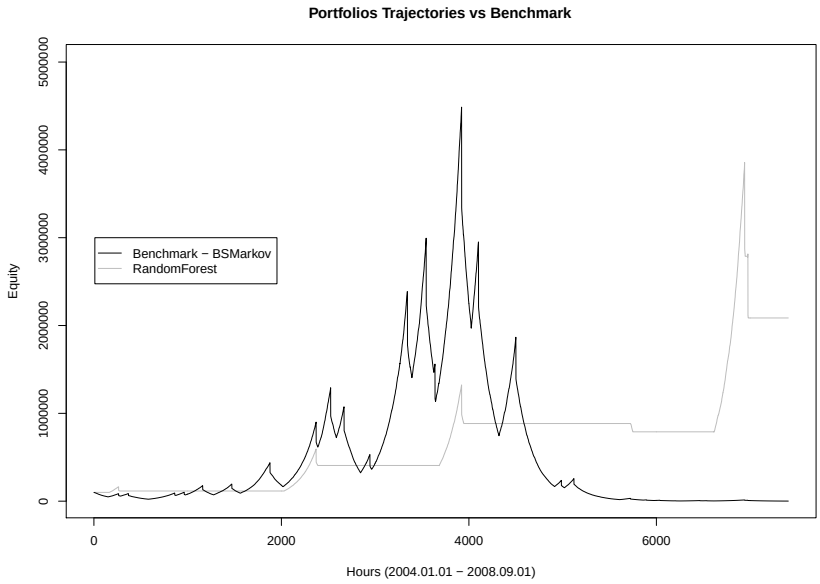

Fig. 5: Algorithm portfolio market value vs benchmark (generated model) portfolio market value.

The simulation results are illustrated in Fig. 5. Variable Y models volume in the observation matrix for the classifier. MSE of predicted out-of-sample values $\tilde{H}^n_{ri}$ and $\tilde{L}^n_{ri}$ was equal 0.01 and 0.013 which are quite satisfactory results in comparison with the average four days (28 hours) high and low price movements given by (18) and (17), i.e. $MSE_{(H^n_{ri}, C)} = 0.08$, $MSE_{(L^n_{ri}, C)} = 0.11$.

Despite of good performance in terms of MSE, the system failed to deduce that state transitions of variable Y proceed dramatic price fall. Note that we have trained the forest in the window of 1000 hours length and the trend turns where rare phenomena during this period. It means that even a large but rare system prediction mistake has only a little impact on the regression absolute error measure of the random forest. Introducing some penalty factor for large prices downfall missclassifications seems to be a proper solution of this problem.

6 Conclusions

In this paper we presented a novel architecture of the system for automated stock trading and verified its performance on the large data set applying walk forward methodology. Although the system failed to generated profitable trading strategy it proved its ability of on-line adaptation to non-stationary time series. Hence we may conclude that application of machine learning techniques for on-line adaptation to real-life data models which distributions change drastically over time seems to be promising.

The major purpose of this work was to give an indication if machine learning techniques are able to generate profitable trading strategy. Although the trading results are not satisfactory there are some further issues worth examining. Firstly, one could try to incorporate information from other past prices and indicators. Secondly, it is worth of interest to examine other soft computing methods as a core of the system such as proposed in [6]. In particular, some clustering abilities of fuzzy sets may be useful for mining profitable trading rules.

References

1. Elder A., Trading for living, Wiley Finance (1993)
2. Ernst P. Chan, Quantitive Trading. Wiley Trading (2008)
3. Fong S., Si Y.W., Tai J., Trend following algorithms in automated derivatives market trading, Expert Systems with Applications 39, pp. 11378-11390 (2012)
4. Hastie T., Tibshirani R., Friedman J., The Elements of Statistical Learning. Springer-Verlag (2009)
5. Jangmin O., Jangwoo Lee J., Jae Won L., Byoung-Tak Z.), Adaptive stock trading with dynamic asset allocation using reinforcement learning. Information Sciences 176, pp. 2121-2147 (2006)
6. Kastner M., Villman T.,, Fuzzy Supervised Self-Organized Map for semi-supervised vector quantization. Lecture Notes in Artificial Intelligence, 7267, pp. 256-265 (2012)
7. Ładyżyński P., Grzegorzewski P., Soft methods in trend detection, In: Combining Soft Computing and Statistical Methods in Data Analysis, Borgelt C. et all. (Eds.), pp. 395-402, Springer (2010)
8. Muh-Cherng W., Sheng-Yu L., Chia-Hsin L., An effective application of decision tree to stock trading. Expert Systems with Applications 31, 270-274 (2006)
9. Qing-Guo W., Jin L., Qin Q., Shuzhi Sam G., Linear, Adaptive and Nonlinear Trading Models for Singapore Stock Market with Random Forests, 2011 9th IEEE International Conference on Control and Automation (ICCA) (2011)

View publication stats